

claude-*analytic*

Tokens ran out faster than expected? Don't know where they went? This tool shows you — locally, in seconds.

```
$ npx github:onsaurav/claude-analytic
```

Or install globally: `npm install -g claude-analytic`, then run `claude-analytic`.

01 — PREMISE

What this *actually* does.

A small CLI that reads your local Claude Code sessions and reports token usage and cache hit rate, so you can see why your quota disappeared. **No network calls. No API keys. Everything stays on your machine.**

Claude Code stores every session as JSONL files under `~/.claude/projects/`. The tool reads those files, tallies tokens by type, computes your cache hit rate, and prints a summary. That's it.

02 — VOCABULARY

Two ideas *worth knowing*.

Two terms drive everything below: the **token** (the unit you're billed for) and the **cache hit** (the mechanic that makes long sessions cheap). Internalize these and the rest of the report reads itself.

The token

The smallest unit of text Claude reads or writes — roughly $\frac{3}{4}$ of a word, or about 4 characters in English. Every prompt you send and every reply Claude returns is measured in tokens, and that's what you're billed for.

"Hello, world!" → about 4 tokens · 500-word message → ~650 tokens · medium source file → 1,000–3,000 tokens.

TYPE 01

Input

TYPE 02

Output

Fresh text you sent to Claude — text the model has to read from scratch, not pulled from the cache. \$3.00 / 1M	Text Claude generated back to you. Five times more expensive than input — generation is the costly half of the loop. \$15.00 / 1M · 5×
TYPE 03 Cache write Input that was stored into the prompt cache so it can be reused on future turns without being re-processed. \$3.75 / 1M	TYPE 04 Cache read Input served from the cache instead of being re-processed. Ten times cheaper than fresh input — this is where savings live. \$0.30 / 1M · 10× cheaper

The cache hit

When you keep talking in the same session, Claude doesn’t re-read your earlier messages from scratch — it pulls them from a prompt cache. Each time the cache is reused, that’s a cache hit.

A higher hit rate means you’re reusing context efficiently and paying less. Cache hit rate = cache reads ÷ (cache reads + fresh input).

03 — DIAGNOSIS

Where your tokens *actually went*.

Sometimes tokens vanish well before you expected. The usual suspects, ordered by frequency:

WHAT HAPPENED	WHY TOKENS DRAINED
Big code generation or refactor	Output tokens cost 5× input — generation adds up faster than people expect.
You ran /clear often	Each clear kills the cache. The next prompts pay full price until the cache warms back up.
Hopping between many projects	Each project has its own cache. None of them stay warm.
Long idle gaps between turns	Cache expires. You pay the warm-up again.
Edited a file Claude read early on	Prefix changed. Everything after the change cache-misses.

Reading your hit rate

A day spent at 90% versus 30% can be a 3–5× cost difference. The number itself tells the story:

85%+

EXCELLENT

60–85%

ROOM TO IMPROVE

< 50%

PAYING FULL PRICE

04 — OUTPUT

What you'll *see*.

Run the command and you get two sections: your top spenders, then the totals. Sample output below.

Claude Analytic Report

Detected Claude path: C:\Users\you\.claude\projects
Total sessions: 3

► Top 5 Sessions by Token Usage

Session	In	Out	Cache Hit
d--workspace-claude...	153	28,150	99.99%
d--workspace-c160c1...	69	21,284	99.99%
d--workspace-claude...	50	12,826	99.99%

► Overall Summary

Metric	Value
Total Input Tokens	272
Total Output Tokens	62,260
Total Cache Read Tokens	4,123,997
Total Cache Write Tokens	135,468
Overall Cache Hit Rate	99.99%

💡 Tip: High cache hit rate = lower cost

Top 5 Sessions by Token Usage — your biggest spenders, ranked, with their individual hit rate.

Overall Summary — totals for every token type and your overall hit rate. Reference pricing per 1M tokens: input **\$3.00** · output **\$15.00** · cache read **\$0.30** · cache write **\$3.75**.

05 — PRACTICE

Three habits that *save tokens*.

01 Stay in one session. Every `/clear` is a cold cache. Resume work where you left it whenever you can.

02 Put stable context first. The cache matches a prefix — anything that changes early invalidates everything that follows. Stable instructions and unchanging files belong at the top.

03 Don't ping-pong between projects. Each project has its own cache. Quick context-switches in short bursts mean none of them stay warm long enough to pay off.

PRIVACY

Everything runs locally. No data leaves your machine. No telemetry, no API keys, no network calls.

INSTALL

```
npx github:onsaurav/claude-analytic  
npm install -g claude-analytic
```

LICENSE

MIT.

— a small tool, kept honest.